

Chapter 7

Guidelines for Good Stories

At this point, with a good foundation of what stories are, how to trawl for and write them, how to identify key user roles, and the role of acceptance testing, we turn our attention to some additional guidelines for writing good stories.

Start with Goal Stories

On a large project, especially one with many user roles, it is sometimes difficult to even know where to begin in identifying stories. What I've found works best is to consider each user role and identify the goals that user has for interacting with our software. For example, consider the Job Seeker role in the BigMoney-Jobs example. She really has one top priority goal: find a job. But we may consider that goal to comprise the following goals:

- search for jobs she's interested in (based on skill, salary, location, and so on)
- automate the search process so she doesn't have to search manually each time
- make her resume available so that companies may search for her
- easily apply for any jobs she likes

These goals (which really are high-level stories themselves) can then be used to generate additional stories as needed.

Slice the Cake

When faced with a large story, there are normally many ways of breaking it into smaller pieces. The first inclination of many developers is to split the story

along technical lines. For example, suppose the team has decided that the story “A Job Seeker can post a resume” is simply too large to fit in the current iteration and must be split. The developers may want to split it along technical boundaries, such as:

- A Job Seeker can fill out a resume form.
- Information on a resume form is written to the database.

In this case, one story would be done in the current iteration while the other story would be deferred until (presumably) the next iteration. The problem with this is that neither story on its own is very useful to users. The first story says that job seekers can fill out a form but that the data is not saved. Not only is this not useful, it would actually waste users’ time. The second story says that the data collected on the form will be written to the database. Without a story to present the form to users, the second story is not useful.

A far better approach is to write the replacement stories such that each provides some level of end-to-end functionality. Bill Wake (2003a) refers to this as “slicing the cake.” Each story must have a little from each layer. This leads to splitting “A Job Seeker can post a resume” like this:

- A Job Seeker can submit a resume that includes only basic information such as name, address, education history.
- A Job Seeker can submit a resume that includes all information an employer may want to see.

Stories that represent a full slice of cake are to be preferred over those that do not. There are two reasons for this. First, exercising each layer of an application’s architecture reduces the risk of finding last minute problems in one of the layers. Second, although not ideal, an application could conceivably be released for use with only partial functionality as long as the functionality that is included in the release slices all the way through the system.

Write Closed Stories

Soren Lauesen (2002) introduces the idea of closure for tasks in his compendium of requirements techniques. His ideas are equally applicable to user stories. A closed story is one that finishes with the achievement of a meaningful goal and that allows the user to feel she has accomplished something.

For example, suppose the BigMoneyJobs website project includes the story “A recruiter can manage the ads she has placed.” This is not a closed story: Managing the ads she’s placed is not something that is ever completely done. Instead, it is an ongoing activity. This story can be better constructed as a set of closed stories, such as:

- A recruiter can review resumes from applicants to one of her ads.
- A recruiter can change the expiration date of an ad.
- A recruiter can delete an application that is not a good match for a job.

And so on. Each of these closed stories is a part of the original story that was not closed. After completing one of these closed stories, a user is likely to feel a sense of accomplishment.

The desire to write closed stories has to be tempered against competing needs. Remember that stories also need to be small enough to be estimatable and small enough to be conveniently scheduled into a single iteration. But stories must also be large enough that you avoid capturing details about them any earlier than necessary.

Put Constraints on Cards

Newkirk and Martin (2001) recommend a practice I’ve found useful. They introduce the practice of annotating a story card with “Constraint” for any story that must be obeyed rather than directly implemented. An example can be seen in Story Card 7.1.

The system must support peak usage of up to 50 concurrent users.

Constraint

- Story Card 7.1 An example of a constraint story card.

Other examples of constraints are:

- Do not make it hard to internationalize the software if needed later.
- The new system must use our existing order database.

- The software must run on all versions of Windows.
- The system will achieve uptime of 99.999%.
- The software will be easy to use.

Even though constraint cards do not get estimated and scheduled into iterations like normal cards, they are still useful. Minimally, constraint cards can be taped to the wall where they act as reminders. Even better, acceptance tests can be written to ensure the constraint is not violated. For example, it would not be difficult to write a test for Story Card 7.1. Ideally the team would write this test during one of the first iterations when there's little chance of it being violated. The team would then continue running the test as part of each subsequent iteration. Whenever possible (and it usually is), write automated tests to ensure that constraints are being met.

For more on constraints as a way of specifying nonfunctional requirements see Chapter 16, "Additional Topics."

Size the Story to the Horizon

You want to focus your attention on the areas that most need it. Usually, this means paying more attention to things happening in the near future than to things happening further out. With stories, you do this by writing stories at different levels based on the implementation horizon of the stories. This means, for example, that stories for the next few iterations would be written at sizes that can be planned into those iterations, while more distant stories could be much larger and less precise. For example, suppose at the highest level we've determined that the BigMoneyJobs website will include four stories:

- A Job Seeker can post a resume.
- A Job Seeker can search job openings.
- A Recruiter can post a job opening.
- A Recruiter can search resumes.

The customer has decided that the first iterations will focus on allowing users to post resumes. Only after the bulk of the resume posting functionality has been added will attention be turned to searching for jobs, posting job openings, and searching resumes. This means that the project team and customer will start having the conversations about "A Job Seeker can post a resume."

That story will be expanded as details are discovered through those conversations; the other three high-level stories will be left alone. A possible list of stories then becomes:

- A Job Seeker can add a new resume to the site.
- A Job Seeker can edit a resume that is already on the site.
- A Job Seeker can remove her resume from the site.
- A Job Seeker can mark a resume as inactive.
- A Job Seeker can mark a resume as hidden from certain employers.
- A Job Seeker can see how many times her resume has been viewed.
- ... and so on about posting resumes...
- A Job Seeker can search job openings.
- A Recruiter can post job openings.
- A Recruiter can search resumes.

In writing your stories, take advantage of the flexibility of stories to be useful at various levels.

Keep the UI Out as Long as Possible

One of the problems that has plagued every approach to software requirements has been mixing requirements with solution specification. That is, in stating a requirement, a solution is also either explicitly stated or implied. Most commonly this happens with aspects of the user interface. You want to keep the user interface out of your stories as long as possible. For example, consider Story Card 7.2, which is a story from a real system. If this story is to be developed early in the life of the project, it includes too many user interface details. Readers of this story are told about the print dialog, printer lists, and at least four ways of searching.

Eventually it will become inevitable for user interface details to slip into stories. This happens as the software becomes more and more complete, and stories shift away from being entirely new functionality to being modifications or extensions of existing functionality.

For example, consider the story “A user can select dates from a date widget on the search screen.” This story may represent three days of work regardless of

whether it's done at the start or end of the project. However, it is not a story you'd expect to have at the start of the project before the user interface has even been considered.

Print dialog allows the user to edit the printer list. The user can add or remove printers from the printer list. The user can add printers either by auto-search or manually specifying the printer DNS name or IP address. An advanced search option also allows the user to restrict his search within specified IP addresses and subnet range.

■ Story Card 7.2 A card with too much user interface detail.

Some Things Aren't Stories

While user stories are a very flexible format that works well for describing much of the functionality of many systems, they are not appropriate for everything. If you need to express some requirements in a form other than user stories, then do so. For example, user interface guidelines are often described in documents with lots of screen captures. Similarly, apart from any user stories, you may want to document and agree upon an interface between important systems, especially if one is being developed by an external vendor.

If you find that some aspect of a system could benefit from expression in a different format, then use that format.

Include User Roles in the Stories

If the project team has identified user roles, they should make use of them in writing the stories. So instead of writing "A user can post her resume" they write "A Job Seeker can post her resume." The difference is minor but writing stories in this way keeps the user in the forefront of the developer's mind. Instead of thinking of bland, faceless, interchangeable users, the developer will begin thinking of real, tangible users whom she needs to satisfy with the software.

Connextra, one of the early adopters of Extreme Programming, incorporated roles into their stories by using a short template. Each story was written in the following format:

I as a (role) want (function) so that (business value)

You may want to experiment with this template or with one of your own. A template like this can help distinguish important from frivolous stories.

Write for One User

Stories are generally most readable when written for a single user. For many stories, writing for one or many users will not make a difference. However, for some stories the difference can be significant. For example, consider the story “Job Seekers can remove resumes from the site.” This could be interpreted to mean that one Job Seeker can remove her own resume and possibly the resumes of others.

Normally, this type of issue will become clear when you think about a story with a single user in mind. For example, the story above could be written as “A Job Seeker can remove resumes.” When written this way, the problem of one Job Seeker removing resumes of others becomes more apparent, though, and the story can be further improved to “A Job Seeker can remove her own resumes.”

Write in Active Voice

User stories are easier to read and understand when written in active voice. For example, rather than saying “A resume can be posted by a Job Seeker” say “A Job Seeker can post a resume.”

Customer Writes

Ideally the customer writes the stories. On many projects the developers help out, either by doing the actual writing during an initial story writing workshop or by suggesting new stories to the customer. But, responsibility for writing stories resides with the customer and cannot be passed to the developers.

Additionally, because the customer is responsible for prioritizing the stories that will go into each iteration, it is vital that the customer understand each story. The best way to do this is to write them.

Don't Number Story Cards

The first time we use story cards many of us are tempted to number them. The usual reasoning is that this will help keep track of individual cards or add some level of traceability to stories. For example, when we discover that the story on card 13 is too large we rip up card 13 and replace it with cards 13.1, 13.2, and 13.3. However, numbering story cards adds pointless overhead to the process and leads us into abstract discussions about features that need to be tangible. I'd rather talk about "the story to add user groups" than "story 13." I especially don't want to talk about "story 13.1."

If you feel compelled to number story cards, instead try adding a short title to the card and use the title as shorthand for the rest of the story text.

Don't Forget the Purpose

Don't forget that the main purpose of a story card is to act as a reminder to discuss the feature. Keep these reminders brief. Add the detail you need to remember where to resume a conversation, but do not replace the conversation by adding more detail to the story card.

Summary

- To identify stories, start by considering the goals of each user role in using the system.
- When splitting a story, try to come up with stories that cut through all layers of the application.
- Try to write stories that are of a size where the user feels justified in taking a coffee break after completing the story.
- Augment stories with other requirements gathering or documenting techniques as necessary for the project's domain and environment.

- Create constraint cards and either tape them to a shared wall or write tests to ensure the constraints are not violated.
- Write smaller stories for functionality the team will implement soon, and write broad, high-level stories for functionality further into the future.
- Keep the user interface out of the stories for as long as possible.
- When practical, include the user role when writing the story.
- Write stories in active voice. For example, say “A Job Seeker can post a resume” rather than “A resume can be posted by a Job Seeker.”
- Write stories for a single user. Instead of “Job Seekers can remove resumes” write “A Job Seeker can remove her own resumes.”
- Have the customer, rather than a developer, write the stories.
- Keep user stories short, and don’t forget their purpose as reminders to hold conversations.
- Don’t number story cards.

Questions

- 7.1 Assume the story “A Job Seeker can search for open jobs” is too large to fit into one iteration. How would you split it?
- 7.2 Which of these stories is appropriately sized and can be considered a closed story?
 - a A user can save her preferences.
 - b A user can change the default credit card used for purchases.
 - c A user can log on to the system.
- 7.3 What simple changes could improve the story “Users can post their resumes”?
- 7.4 How would you test the constraint “The software will be easy to use”?

This page intentionally left blank

PART II

Estimating and Planning

With an understanding of stories under our collective belts, we can turn our attention to estimating and planning projects with user stories. On almost all projects, we need, or are asked, to provide an estimate of how long the project will take. Marketing campaigns need to be readied, users need to be trained, hardware needs to be purchased and so on. Each of these activities depends upon a project's plans.

In Part II, we will see how to estimate the stories and how to create a high-level release plan for delivery of the highest priority stories. Then we'll see how the release plan is refined with additional detail at the start of each iteration with the additional planning necessary to perform the work of that iteration. Finally, we'll look at ways to measure and monitor the progress of the project so that we can constantly adjust the plan to reflect the knowledge we gain from each iteration.

This page intentionally left blank